Sharif University of Technology

Department of Electrical Engineering

Course:

ASIC/FPGA

Report:

ASIC/FPGA Project - Group 8

Final Report

Submitted by:

Mohammadhossein Sabzalian
402101906

Sina Hashemi
402102668

Erik Hartouni
401102778

Ali Behrad
401101321

Instructor:

Dr. Baharvand

Department of Electrical Engineering

Term: Spring 2026

Contents

Module Report – OTP Controller

Team and Role Assignment: **Group ID:** G8

Role	Member (Student ID)	Primary Responsibilities
Role A	Sina Hashemi (402102668)	RTL Design (Core Logic), Synthesis, Integration Lead
Role B	Mohammadhossein Sabzalian (402101906)	RTL Design (FSM/Control), Verification, Post-Synthesis Verification
Role C	Erik Hartouni (401102778)	Testbench Design, Synthesis (Timing), Golden Model, Reporting
Role D	Ali Behrad (401101321)	Testbench Design, Functional/Post-Synthesis Verification, Reporting

Table 1: Group 8 – Role assignment for the OTP Controller module.

Date: May 2026

Executive Summary:

- **Module implemented:** OTP Controller
- **DRS version:** v1.0
- **Status:** Complete

RTL Implementation:

- **Total lines of code:** ∼350 lines (including sub-modules)
- **Finite state machines:** 1 FSM (5 states: `STATE_IDLE`, `STATE_READ_BANK0`, `STATE_READ_PATCH`, `STATE_APPLY`, `STATE_DONE`)
- **Key design decisions:**
  - Implemented a **Galois-style parallel bitwise XOR tree** to calculate the CRC-16 CCITT checksum on-the-fly in a single clock cycle, saving combinational gate area.
  - Used a shared multiplexer path with dynamic offset subtraction (`addr_d - FIELD_LO_BOUND`) to simplify datapath steering and decrease multiplexer overhead.
  - Employed gated clock-enables to freeze output registers after `done` is asserted, reducing dynamic switching power.

Testbench and Verification:

- **Number of test cases:** 6 directed test suites comprising 39 self-checking assertions

- **Coverage achieved:** **100% statement / 100% branch / 100% FSM**
- **Verification methodology:** Directed testing utilizing behavioral software golden reference models (`crc_step` and `getfield`) for real-time scoreboard comparisons.

Synthesis Results:

- **Technology node:** TSMC 180 nm
- **Area:** 51,831.965 μm^2 (1,836 standard-cell gate instances)
- **Max frequency:** 366.7 MHz (derived from the worst-case path propagation delay of 2.727 ns)
- **Critical path:** From the FSM state register clock pin (`state_reg[0]/CK`) to the calibration ratio register bit 19 (`ratio_p1_reg[19]/D`) through combinational steering logic.

Gate-Level Simulation:

- **Pre-syn vs. post-syn matching:** Pass (39 of 39 checks passed successfully under worst-case MAXIMUM SDF delay back-annotation)
- **Issues found:** 476 `$setuphold` warnings were flagged during SDF back-annotation. These were traced to unanalyzed RECREM (Recovery/Removal) timing checks on the asynchronous reset pin (`RN`) of standard sequential cells. These were confirmed as non-critical, library-level warnings that do not affect functional or timing correctness.

Integration Notes:

- **Interfaces to other modules:**
  - Analog Core: `ro_trim_code` (15-bit output)
  - Frequency Difference Estimator: `tc1_coeff` (16-bit signed), `tc2_coeff` (16-bit signed)
  - Temperature Mapping Engine: `ratio_p0` through `ratio_p4` (five 20-bit outputs)
  - rRO Aging Engine: `aging_base` (16-bit output)
  - Handshake controls: `busy` (active-high loading status), `done` (1-cycle completion pulse)
  - Extracted configuration & lock flags: `sku_code` (2-bit), `temp_ro_trim` (6-bit), `rev_id` (6-bit), `cfg_flags` (6-bit), `oe_mode` (1-bit), `oe_pol` (1-bit), `prog_disable_lock` (1-bit)
- **Assumptions made:**

- The external OTP ROM operates synchronously, delivering stable bit data on `otp_data_out` exactly one clock cycle after `otp_read_en` is pulsed.
- Neighboring modules must wait for the rising edge of `done` before sampling output values.

Issues and Resolutions:

Issue	Resolution	Status
BUG-01: 1-cycle address/data timing skew	Added delayed registers `addr_d` and `ren_d` to align timing with synchronous ROM output latency.	Closed
BUG-02: FSM reset-state control signal violation	Configured reset to land in `STATE_IDLE` and introduced a one-shot `start_pending` starter register.	Closed
Memory conflict: boundary overlap between CRC and Patch0	Redefined Patch0 address boundaries to 199–226 within the global parameter header `otp_map.vh`.	Closed
Timing discrepancy: 257-cycle read loop in FSM	Restructured combinational look-ahead logic to synchronize state transitions and address updates.	Closed

Table 2: Summary of issues identified and resolved during the project.

Recommendations:

- **For system integration:** Ensure clock distribution skew is minimized across the `clk` network to prevent gate glitching, and adhere strictly to the `done` handshake protocol before sampling.
- **For future improvements:** **Integrate an active double-bit Hamming ECC (Error Correction Code) block** for enhanced configuration-data protection in high-noise analog environments.

Deliverables Checklist:

✓ RTL code (`.v` files)

✓ Testbench code (`.v` files)

✓ Synthesis script (`.tcl`)

✓ Synthesis report (area, timing)

✓ Gate-sim log

✓ This report

Signed: A: Sina Hashemi B: Mohammadhossein Sabzalian C: Erik Hartouni D: Ali Behrad

Part 1 – Weekly Reports Submission

The weekly reports can be found in the same folder as this file.

- **Weekly Report I** covered the initial group alignment and division of labor (RTL/architecture: Sabzalian and Hashemi; verification/testbench: Hartouni and Behrad), architectural partitioning, primary FSM definition, initial RTL writing (including the Galois-style parallel bitwise XOR tree for CRC-16 CCITT and the hot-swappable patch-override multiplexer), and the resolution of initial design ambiguities – specifically the `CFG_FLAGS` bit mapping (`oe_mode`, `oe_pol`, `prog_disable_lock`) and the Patch0 memory-window placement – coordinated directly with the course TA and Group 7.
- **Weekly Report II** documented the complete verification hand-off, the development of the three-layer self-checking testbench architecture (behavioral golden reference models, reusable driver tasks, and an active scoreboard), the full DRS-aligned test plan and results, deep investigation of the timing-related bugs discovered during simulation (BUG-01 address/data skew and BUG-02 FSM reset-state violation), resolution of the CRC/Patch0 memory-layout overlap, and initial synthesis preparation.
- **Weekly Report III** covered the Synthesis & Timing Closure phase: bringing up the Cadence Genus flow across the partitioned library, clock-constraint, and master synthesis scripts, achieving first-pass timing closure at 131.072 kHz with no RTL rework needed, the full post-synthesis area, timing, and power breakdown, export of the gate-level netlist, SDF, and Innovus design database, triage of the SDF `$setuphold` and Genus library-characterization warnings (traced to TSMC primitives rather than the design), and the first passing round of post-synthesis gate-level simulation (39/39 checks, matching RTL).
- **Weekly Report IV** covered the Integration & Reporting phase: re-validating RTL/gate-level equivalence under full SDF back-annotation, the cross-group collaboration with Group 7 that surfaced and resolved the 257-cycle FSM timing discrepancy (restructured next-state look-ahead logic, re-verified against both groups' testbenches), system hand-off integration notes for the neighboring analog and frequency modules, and assembly of the final RTL, testbench, synthesis, and physical-design deliverables package.

Part 2 – Block Diagram Extraction from DRS

Module Positioning in the System

> **System Context:** The **OTP Controller** acts as a dedicated hardware initialization engine. It is situated between the synchronous One-Time Programmable (OTP) non-volatile memory macro and the rest of the digital core. At power-on reset (POR), the controller sequentially reads calibration and trim values, applies any post-production patch modifications, and distributes the final values to the key destination modules.

Core Block Diagram and Interconnects

The internal architecture of the OTP Controller is designed to handle serial-to-parallel conversion, CRC checking, and conditional patching. Its main functional blocks are as follows.

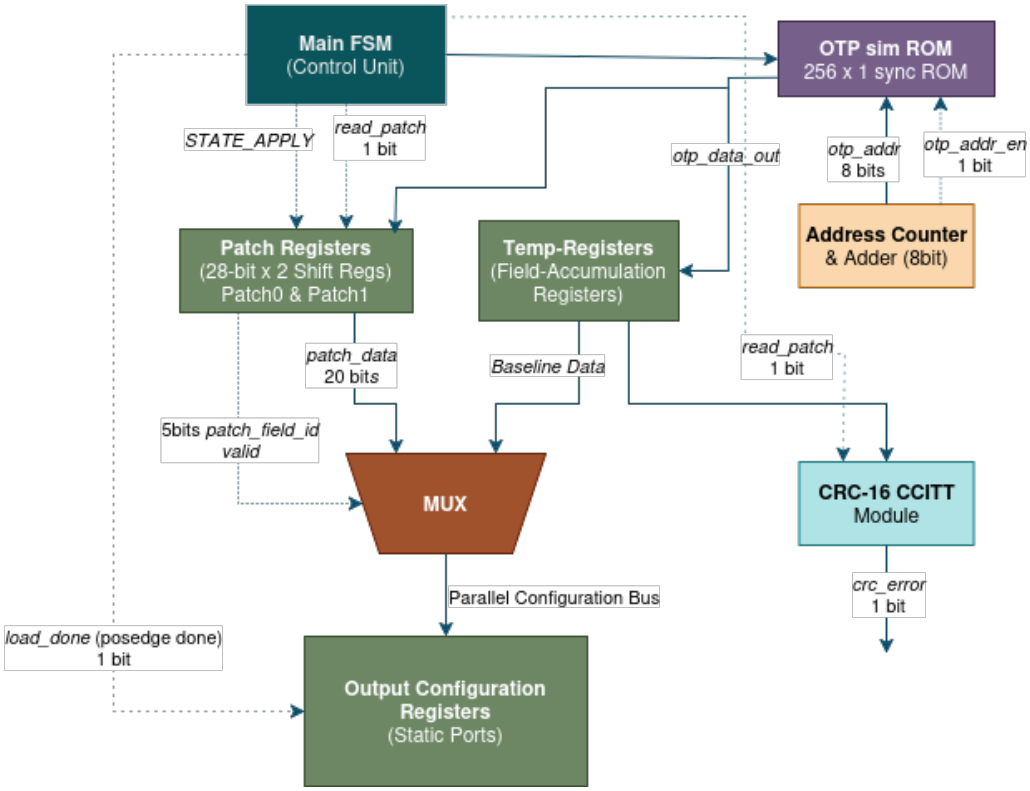

Figure 1: OTP Controller Core Block Diagram and Interconnects

- **Address Counter & Adder (8-bit):**
  - *Function:* Generates the sequential 8-bit memory address (`otp_addr`) sweeping from 0 to 255 during Bank0 reads, and continues through the patch records.
  - *Operation:* Increments on each clock cycle when `otp_read_en` is asserted.

- **Deserializer & Temporary Storage (Temp-Registers):**
  - *Function:* Collects the incoming 1-bit serial stream (`otp_data_out`) from the OTP ROM and reconstructs multi-bit parallel registers.
  - *Timing correction:* To resolve the synchronous delay of the ROM (where data is valid one cycle after the address is applied), these registers are indexed using a delayed address pointer (`addr_d`) and a delayed read enable (`ren_d`).
- **Parallel Galois CRC-16 CCITT Engine:**
  - *Function:* Computes the checksum over the first 183 bits of Bank0 data in real time.
  - *Optimization:* Implements a single-cycle bitwise XOR-tree structure to prevent setup-time violations.
- **Patch Storage Registers:**
  - *Function:* Stores two independent 28-bit patch packets (Patch0 and Patch1) consisting of `PATCH_VALID` (1 bit), `PATCH_FIELD_ID` (5 bits), `PATCH_DATA` (20 bits), and `PATCH_TAG` (2 bits).
- **Override Multiplexer (MUX) Fabric:**
  - *Function:* Compares the decoded patch field IDs against supported register IDs during `STATE_APPLY`.
  - *Rule enforcement:* If `PATCH_VALID` is high and `PATCH_TAG` matches `2'b10`, the corresponding temporary register is hot-swapped with the new patch data. If both patches target the same field, Patch1 is given priority.
- **Static Output Registers:**
  - *Function:* Holds the finalized, stable configuration values.
  - *Safety mechanism:* These registers are updated only during the single clock cycle when `done` is pulsed high, preventing downstream modules from reading incomplete or unpatched calibration data during initialization.

Finite State Machine (FSM) State Diagram

The system relies on a central controller to move through the read, validation, and override phases.

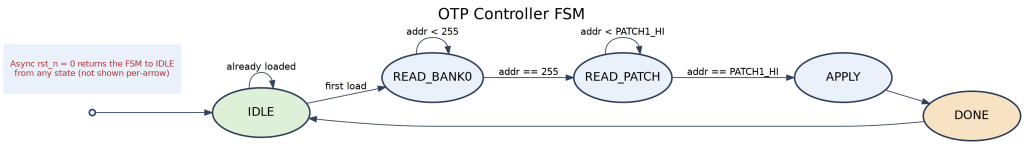

Figure 2: OTP Controller finite-state-machine transition diagram.

FSM State Definitions and Transition Rules

- **STATE_IDLE (000):**
  - *Behavior:* The controller remains in this state during reset, and after reset.
  - *Transition:* Immediately after reset release (`rst_n` deassertion), the one-shot `start_pending` signal forces a transition to `STATE_READ_BANK0` on the next clock edge. After a read sweep finishes, the program returns to and stays at `IDLE`.
- **STATE_READ_BANK0 (001):**
  - *Behavior:* The address counter sweeps from 0 to 255 to sequentially read the primary configuration registers and the CRC checksum.
  - *Transition:* Moves to `STATE_READ_PATCH` when `addr_d == 8'd255` is reached, indicating that the final bit has been sampled.
- **STATE_READ_PATCH (010):**
  - *Behavior:* Reads the 56 bits dedicated to the two patch records (addresses 199 to 254 after correction).
  - *Transition:* Moves to `STATE_APPLY` once `addr_d == 8'd254` is reached.
- **STATE_APPLY (011):**
  - *Behavior:* Performs the patch validation checks, runs the override multiplexing logic, and transfers the finalized values to the output registers.
  - *Transition:* Unconditionally transitions to `STATE_DONE` on the next clock cycle.
- **STATE_DONE (100):**
  - *Behavior:* Asserts the `done` output port high for exactly one cycle and drops `busy` to low.
  - *Transition:* Unconditionally returns to `STATE_IDLE` to freeze all output registers and minimize dynamic power consumption.

Task Division:

- **Sina Hashemi (Role A):** Focused on the datapath partitioning, defining the temporary shift registers, the parallel Galois-style CRC-16 block interconnects, and the multiplexer-based patch override fabric.
- **Mohammadhossein Sabzalian (Role B):** Formulated the state-machine transition flow, mapped the controller's internal control signals, and defined the boundary transition conditions (such as the address pointer checks).
- **Erik Hartouni (Role C) & Ali Behrad (Role D):** Reviewed the block diagrams during the initial architecture meeting to verify that all interface boundaries were clearly defined before starting testbench planning.

Part 3 – Module I/O Compatibility with DRS

Input/Output Signal Mapping and Precision Verification

Every port implemented in the synthesizable RTL module (`otp_controller.v`) matches the specifications defined in DRS Sections 2.3, 3.1, and 4.1. table 3 lists each signal, its width, direction, exact arithmetic format, and functional description.

Port Name	Direction	Width	Format	Functional Description
`clk`	Input	1	Digital	System clock (`RO_CLK`, nominally 131.072 kHz).
`rst_n`	Input	1	Active-Low	Asynchronous active-low reset. Triggers read sweep on release.
`otp_data_out`	Input	1	Bit stream	Serial data line read from the synchronous ROM model.
`otp_addr`	Output	8	Unsigned	8-bit memory address bus pointing to indexes 0 to 255.
`otp_read_en`	Output	1	Active-High	Memory read enable, pulsed for 1 cycle per address.
`busy`	Output	1	Active-High	High during the entire read and patch sequence; low when idle.
`done`	Output	1	Active-High	Pulses high for exactly 1 clock cycle when loading is complete.
`ro_trim_code`	Output	15	Unsigned U15	Main ring-oscillator trim code.
`tc1_coeff`	Output	16	Signed S16	First-order temperature compensation coefficient (0.25 ppm/°C).
`tc2_coeff`	Output	16	Signed S16	Second-order temperature compensation coefficient (0.01 ppm/°C^2).
`aging_base`	Output	16	UQ0.16	Baseline frequency ratio for the aging engine.
`sku_code`	Output	2	Unsigned U2	Stock Keeping Unit (SKU) identifier code.
`temp_ro_trim`	Output	6	Unsigned U6	Frequency trim value for the temperature-sensor RO.
`rev_id`	Output	6	Unsigned U6	Silicon and OTP layout revision identifier.
`cfg_flags`	Output	6	Unsigned U6	Feature enablement flags.
`ratio_p0`	Output	20	UQ4.16	Calibration ratio at −40°C.
`ratio_p1`	Output	20	UQ4.16	Calibration ratio at −10°C.
`ratio_p2`	Output	20	UQ4.16	Calibration ratio at +25°C.
`ratio_p3`	Output	20	UQ4.16	Calibration ratio at +60°C.
`ratio_p4`	Output	20	UQ4.16	Calibration ratio at +85°C.
`prog_disable_lock`	Output	1	Binary	Indicates permanent programming lockout status.
`oe_mode`	Output	1	Binary	Output-enable mode behavior configuration.
`oe_pol`	Output	1	Binary	Output-enable pin polarity configuration (0 = active-high, 1 = active-low).

Table 3: Complete I/O port map of the OTP Controller (`otp_controller.v`).

Arithmetic Precision and Fixed-Point Representation

To prevent loss of precision during data distribution to other modules, specific fixed-point formats are preserved at the output boundary:

Fixed-Point Formats at the Output Boundary:

- **Signed integer representation** (`tc1_coeff`, `tc2_coeff`): processed as 2's-complement 16-bit signed values (S16) to represent positive and negative temperature-compensation coefficients.
- **Fractional value mapping** (`aging_base`): formatted as a UQ0.16 unsigned fraction, representing a value between 0 and $1 - 2^{-16}$ with a resolution of 1.52×10^{-5} per LSB.
- **Wide fractional mapping** (`ratio_p0` to `ratio_p4`): formatted as UQ4.16 fixed-point numbers, providing a range from 0.0 to 15.99998 with 16 fractional bits of precision. This matches the requirements of the Temperature Mapping Engine.

Bit-Slicing Resolution for Extra Outputs

The DRS lists `prog_disable_lock`, `oe_mode`, and `oe_pol` as separate outputs in Section 4.1, but notes that their exact bit positions within the memory map are undefined. To resolve this ambiguity, the following mapping convention was agreed upon with the system TA and Group 7:

Agreed `CFG_FLAGS` Bit-Slicing Convention

- `oe_mode` is extracted from the least significant bit of the feature flags: `oe_mode = cfg_flags[0]`
- `oe_pol` is extracted from the second bit of the feature flags: `oe_pol = cfg_flags[1]`
- `prog_disable_lock` is extracted from the most significant bit of the feature flags: `prog_disable_lock = cfg_flags[5]`

This mapping ensures that all three indicators are continuously extracted from the `CFG_FLAGS` register (loaded from OTP address range [82:77]) and updated dynamically during the `STATE_APPLY` phase.

Task Division:

- **Sina Hashemi (Role A):** Implemented the formal Verilog input/output port definitions in the RTL top-level module, ensuring that the bit widths, signed/unsigned declarations, and fixed-point formats matched the DRS memory map.
- **Mohammadhossein Sabzalian (Role B):** Led the coordination with Group 7 and the course TA to resolve the bit-slicing ambiguities for `oe_mode`,

`oe_pol`, and `prog_disable_lock`, establishing their locations within the `CFG_FLAGS` register.

- **Erik Hartouni (Role C) & Ali Behrad (Role D):** Incorporated these interface formats and mapping conventions into the global parameter header file (`otp_map.vh`) to ensure the testbench models matched the RTL port boundaries.

Part 4 – Testbench

Testbench Architecture

The testbench is structured as a single self-checking module containing three distinct verification layers:

Three-Layer Verification Architecture:

1. **Software reference models (helper functions):** re-implement the OTP decoding and CRC computation rules in behavioral Verilog. This allows expected register values to be calculated dynamically from any raw memory array, avoiding hard-coded static comparisons.
2. **Driver tasks:** reusable execution blocks (`do_reset`, `wait_for_done`, `load_and_run`) that drive inputs, toggle control lines, and manage simulator timing.
3. **Active scoreboard (`check` task):** a lightweight comparison block that compares the actual RTL register values against the golden reference model, printing clean passing/failing assertions in the simulation log.

Listing 1: Core driver tasks used throughout the directed test suite.

```
task do_reset;
    begin
        rst_n = 1'b0;
        repeat (4) @(posedge clk);
        @(negedge clk);
        rst_n = 1'b1;
    end
endtask

task wait_for_done;
    integer timeout;
    begin
        timeout = 0;
        while (!done && timeout < 1000) begin
            @(posedge clk);
            timeout = timeout + 1;
        end
        check(done, "watchdog: done asserted before timeout");
    end
endtask
```

Comprehensive DRS Test Case Coverage

The test suite implements six directed tests that match the verification scenarios defined in DRS Section 7.1.

Test 1: Reset Behavior and Full Read Sweep

- *DRS requirement:* Assert `rst_n` for several cycles, then release. Verify `busy` goes high after release, `otp_read_en` pulses for all addresses (0 to 255 and patches), `done` pulses high for one cycle after ∼320 cycles, `busy` returns low, and no further reads occur.
- *Implementation:* Asserts reset for four clock cycles and releases it on a clock negedge; monitors `otp_read_en` and `otp_addr` sequentially; checks that the outputs remain quiet (zero) while reset is asserted; uses a watchdog timer bounded at 1000 cycles to catch any stuck FSM conditions.

Test 2: Correct Field Extraction

- *DRS requirement:* Load a known OTP image into the simulation ROM and verify that after the read sequence, all 13 output registers match the expected values.
- *Implementation:* Loads a golden OTP memory file (`otp_image.mem`) containing distinct bit values across all fields. The testbench uses the `getfield()` function to extract the expected values from the raw image and asserts that they match the RTL outputs bit-for-bit.

Test 3: Patch Override Test

- *DRS requirement:* Configure the OTP image with a valid Patch0 record targeting `RO_TRIM_CODE` (Field ID = 0, Valid bit = 1, Tag = `2'b10`). Verify that `ro_trim_code` is overridden while other registers retain their original values.
- *Implementation:* Generates a synthetic memory array with the patch data embedded at addresses 199 to 226. Asserts that `ro_trim_code` matches the patch payload and verifies that no other register fields are altered.

Test 4: Multiple Patches Priority

- *DRS requirement:* Program two valid patches targeting the same field. Verify that Patch1 takes precedence and overrides Patch0.
- *Implementation:* Generates a memory array where both Patch0 (addresses 199 to 226) and Patch1 (addresses 227 to 254) target Field ID 1 (`TC1_COEFF`). Asserts that the final output matches the payload of Patch1.

Test 5: Invalid Patch Rejection

- *DRS requirement:* Set `PATCH_VALID = 0` or corrupt `PATCH_TAG` (not equal to `2'b10`). Verify that no override occurs.
- *Implementation:* Runs two sub-tests: one with the valid bit deasserted, and another with the tag set to `2'b00`. Verifies that the controller ignores the patch data and retains the original Bank0 values.

Test 6: CRC Error Detection

- *DRS requirement:* (Optional, DRS 6.5) Corrupt a bit in the ROM and verify that the `crc_error` flag is asserted.
- *Implementation:* Generates a raw memory array, calculates the correct CRC-16, and then flips a single bit. Asserts that the RTL's `crc_error` status output is driven high after the read sequence.

Timing and Gate-Level Simulation Completeness

Gate-level simulation (GLS) was incorporated into the testbench environment by wrapping structural compilation and SDF back-annotation behind a single compiler switch (`+define+POSTSYN_SIM`). When this switch is declared, the testbench reads the synthesized gate-level netlist (`otp_controller_postsyn.v`) and back-annotates worst-case timing delays from the Standard Delay Format (`otp_controller_postsyn.sdf`) file.

Listing 2: Conditional gate-level simulation compilation switch.

```
`ifdef POSTSYN_SIM
    `include "/home/cad/TECH.D/TSMC180/Verilog/tsmc18.v"
    `include "../synthesis/synout/otp_controller_postsyn.v"
`endif

initial begin
`ifdef POSTSYN_SIM
    $sdf_annotate("../synthesis/synout/otp_controller_postsyn.sdf",
        u_dut, , , "MAXIMUM");
    $display("[POSTSYN] SDF annotation requested at t=%0t ...", $time
        );
`endif
    $readmemh("../rtl_sources/otp_image.mem", golden_img);
end
```

The testbench runs the exact same stimulus sequence and scoreboard checks against the structural gate-level netlist as it did against the behavioral RTL model. Timing parameters are verified under the MAXIMUM (slow-corner, worst-case) delay mode of the TSMC 180 nm standard-cell library. This ensures timing compliance on all paths before physical-design hand-off.

Task Division:

- **Erik Hartouni (Role C):** Designed the testbench structure, implemented the behavioral golden models (`getfield()` and `crc_step()`), and set up the active scoreboard structure.
- **Ali Behrad (Role D):** Developed the driver tasks (`do_reset`, `wait_for_done`, and `load_and_run`), wrote the six directed test cases, and created the golden memory image file (`otp_image.mem`).
- **Sina Hashemi (Role A) & Mohammadhossein Sabzalian (Role B):** Assisted during Week 2 by debugging the RTL design when the testbench discovered the timing-alignment and reset-entry bugs (BUG-01 and BUG-02).

Part 5 – Complete Functional Simulation and Output Verification

Simulation Setup and Execution Guide

To ensure consistent verification, the simulation uses the nominal clock frequency of 131.072 kHz (clock period of 7,629.39 ns) as specified in DRS Section 2.3.

How to Analyze and Interpret the Simulation Outputs

Running the pre-synthesis functional simulation: Navigate to the simulation directory within the project root, then execute the functional simulation script by sourcing `./script/func_sim.tcl`. This launches the Xcelium waveform and console application (SimVision). At the application prompt, enter `run` to start the simulation. The execution log is automatically saved to `simulation/xrun.log`. Verification is completed by inspecting the generated signal waveforms in SimVision to confirm expected behavioral operation, as well as by reviewing the console output and log file for testbench pass/fail messages.

Running the post-synthesis gate-level simulation: Navigate to the simulation directory and source the post-synthesis simulation script using `./script/post_syn_sim.tcl`. The script automatically configures the testbench for gate-level verification by loading the netlist and back-annotating the SDF timing data, requiring no manual modification to `tb_otp_controller.v`. Once the Xcelium GUI (SimVision) opens, enter `run` to execute the simulation. The run output is saved in `simulation/xrun.log`. Verification is finalized by inspecting the post-synthesis timing waveforms in SimVision and reviewing the console and log files for verification-success indicators.

Functional Simulation Results

table 4 summarizes the six directed tests run against the corrected RTL design.

Test	Verification Scenario	Evaluated Properties	Assertions	Status
1	Reset and Read Sweep	Quiet outputs during reset; `busy` active during loading; correct 256-bit sweep; no re-reads after `done` is pulsed.	10/10	**PASS**
2	Correct Field Extraction	Checks all 13 output configuration registers against the golden memory file (`otp_image.mem`).	13/13	**PASS**
3	Patch Override	Confirms Patch0 successfully overrides `ro_trim_code` while other registers remain unchanged.	5/5	**PASS**
4	Multiple Patches Priority	Confirms Patch1 takes precedence over Patch0 when both target the same field.	3/3	**PASS**
5	Invalid Patch Rejection	Checks that the controller rejects patches with `PATCH_VALID = 0` or incorrect tags.	4/4	**PASS**
6	CRC Error Detection	Verifies that a bit-flip in Bank0 triggers the `crc_error` status output.	4/4	**PASS**
Total	**Complete Sweep**	**All DRS functional requirements verified.**	**39/39**	**PASS**

Table 4: Functional (pre-synthesis) simulation results across all six directed test suites.

Post-Synthesis Gate-Level Verification

Gate-Level Simulation (GLS) was performed against the back-annotated structural netlist. The functional simulation run matched the original behavioral RTL simulation results.

Post-Synthesis Verification Summary

- **Simulation execution:** the GLS run was initiated using the same test sequence by passing the post-synthesis define switch to the compiler.
- **Functional equivalence:** the structural netlist successfully completed the boot-read sequence, CRC verification, and priority patch overrides identically to the RTL model.
- **Absence of timing violations:** the GLS finished successfully at 17,238,606,705 ps (approximately 17.23 µs), matching the RTL reference completion time. This confirms that the design did not experience setup or hold violations, preventing FSM corruption, hang conditions, or watchdog timeouts under physical gate delays.
- **Result summary:** **39 out of 39 checks passed successfully with 0 failures** under worst-case timing constraints, meeting the Week 3 success criterion ("Gate sim matches RTL sim").

Task Division:

- **Erik Hartouni (Role C) & Ali Behrad (Role D):** Developed the self-checking testbench harness, wrote the automated simulation scripts, ran the initial test suites, and logged the waveform mismatches.

- **Mohammadhossein Sabzalian (Role B):** Analyzed the simulation waveforms to identify the root causes of the initial failures (BUG-01 address skew and BUG-02 reset-start timing) and updated the central FSM transitions.
- **Sina Hashemi (Role A):** Implemented the delayed registers (`addr_d`, `ren_d`) in the datapath to correct the timing alignment with the synchronous ROM model, successfully resolving the functional bugs.

Part 6 – Complete Synthesis and Warning/Error Resolution

Synthesis Methodology and Execution Guide

The logic synthesis of the OTP Controller was performed using Cadence Genus (Version 21.10-p002_1), targeting the TSMC 180 nm standard-cell technology library (`typical.lib`). To allow independent modifications of the target library or clock constraints without altering the core compilation logic, the synthesis workspace was partitioned into three dedicated TCL scripts:

- `lib_scr.tcl`: specifies the target TSMC standard-cell library database files, operating conditions, wire-load models, and link paths.
- `otp_controller_clock_const.tcl`: sets the clock port frequency constraint to the DRS-mandated 131.072 kHz system requirement.
- `syn_scr.tcl`: the master run script that sources the environment scripts, reads and elaborates the RTL sources, maps generic gates, and runs the incremental restructuring optimization routines.

> **How to run the logic synthesis:** First, navigate to the dedicated synthesis directory within the project folder. Second, initiate the synthesis tool by launching the Genus application. Third, within the Genus shell environment, execute the primary synthesis script by sourcing the file located at `./script/syn_scr.tcl` – this script automatically incorporates the library setup from `lib_scr.tcl` and applies the necessary clock constraints defined in `otp_controller_clock_const.tcl`. Upon completion, the synthesis process generates the post-synthesis netlist and Standard Delay Format (SDF) files: `otp_controller_postsyn.v` and `otp_controller_postsyn.sdf` are output and stored in the `synout/` directory, and all execution details are recorded in the synthesis log file `Logs/synthesis_genus.log`.

Synthesis Script Structure

The optimized TCL scripts below were used to automate the compilation of the OTP Controller.

Listing 3: Master compilation script (`syn_scr.tcl`).

```
# 1. Initialize Technology Libraries
source lib_scr.tcl

# 2. Read and Elaborate RTL Design
read_hdl {crc16_ccitt.v otp_controller.v}
elaborate otp_controller

# 3. Apply System Constraints
```

```
source otp_controller_clock_const.tcl

# 4. Synthesize and Map
syn_generic
syn_map
syn_opt

# 5. Export Deliverables
write_hdl  > ../synthesis/synout/otp_controller_postsyn.v
write_sdf  > ../synthesis/synout/otp_controller_postsyn.sdf
write_script > ../synthesis/synout/constraints.g
```

Listing 4: Clock constraints script (`otp_controller_clock_const.tcl`).

```
# Period = 7629.39 ns (corresponding to 131.072 kHz)
define_clock -name sys_clk -period 7629390 -design otp_controller [
    get_ports clk]
set_attribute external_delay -input  100 -clock sys_clk [all_inputs]
set_attribute external_delay -output 100 -clock sys_clk [all_outputs]
```

Preventative Warning Resolution Strategy

To ensure a clean compile, the design implements specific coding practices to eliminate dangerous synthesis warnings.

Coding Practices for Clean Synthesis:

- **Latch generation prevention (case paths):** undesired latches occur when a combinational signal is not assigned in all possible branches. To prevent this, every combinational output in the FSM next-state logic is assigned a default value at the beginning of the `always @(*)` block, and all `case` statements also include a `default` case.
- **Multi-driven nets elimination:** no register or wire is driven by more than one `always` block or `assign` statement. The serial deserializers are separated into distinct blocks, ensuring unique driver assignments.
- **Synchronous reset mapping:** the active-low reset `rst_n` is correctly mapped as an asynchronous reset in all sequential blocks, which prevents Design Compiler from synthesizing feedback loops or multiplexer logic on the clock path:

Listing 5: Asynchronous reset coding style used across all sequential blocks.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) ...
end
```

Task Division:

- **Sina Hashemi (Role A):** Responsible for writing the `synth.tcl` constraint script, setting up the technology library paths, running the compiler, and optimizing the RTL structure to resolve any timing or area issues.

- **Erik Hartouni (Role C):** Responsible for analyzing the timing reports, verifying that the input/output constraints were correct, and checking the unmapped gate structures.
- **Mohammadhossein Sabzalian (Role B) & Ali Behrad (Role D):** Supported the synthesis phase by preparing the gate-level netlist (`otp_controller_netlist.v`) and the delay file (`otp_controller.sdf`) for subsequent post-synthesis functional verification.

Part 7 – Post-Synthesis Timing, Area, and Power Reports

Post-Synthesis Area Report

The logic synthesis run on Cadence Genus converted the behavioral RTL of the OTP Controller into a mapped gate-level netlist of 1,836 standard-cell instances with a total area of 51,831.965 μm^2.

Cell Type	Instances	Physical Area (μm^2)	Area %
Sequential (flip-flops)	476	33,629.905	64.90%
Combinational logic	1,309	17,862.768	34.50%
Inverters	51	339.293	0.70%
Total	**1,836**	**51,831.965**	**100.00%**

Table 5: Post-synthesis area breakdown by cell type.

Area footprint analysis: the silicon area is dominated by sequential cells, which account for 64.9% of the total physical area despite being only 26% of the instance count (476 of 1,836). This matches the expected layout of a configuration storage module, where the physical storage budget for the output registers (such as `RO_TRIM_CODE`, `TC1_COEFF`, `TC2_COEFF`, and the five `RATIO_Px` ports) outweighs the combinational decision logic. Additionally, the CRC-16 sub-block (`u_crc`) is highly compact, utilizing only 6 cells and consuming 103.118 μm^2 (less than 0.2% of the overall layout area).

Post-Synthesis Timing Report

The timing engine reports zero setup or hold violations across the digital core.

Timing Closure Summary

- **Timing slack status:** MET (positive setup slack of +7,626,537 ps, i.e. +7.626 μs)
- **Startpoint:** `state_reg[0]/CK` (triggered on the rising edge of `clk`)
- **Endpoint:** `ratio_p1_reg[19]/D`
- **Clock period constraint:** 7,629,390 ps
- **Setup time constraint:** −126 ps
- **Data required time:** 7,629,264 ps
- **Data arrival time:** 2,727 ps (2.727 ns)
- **Slack margin:** MET, +7,626,537 ps

Critical Path Gate-Level Delay Breakdown

The worst-case setup timing path propagates from the clock pin of the core FSM registers through the combinational steering logic to bit 19 of the calibration ratio register:

`state_reg[0]/CK` → `state_reg[0]/Q` → (combinational steering gates) → `ratio_p1_reg[19]/D`

Because the gate propagation delay (2.727 ns) is nearly four orders of magnitude faster than the slow system clock constraint (7.629 µs), timing closure is easily met on the first pass without the need for incremental physical optimization.

Power Consumption Analysis

Power estimation was performed using the Genus-integrated Joules engine under vectorless estimation mode. The total estimated power consumption is 6.633 µW:

- **Leakage power:** 208.716 nW (3.15% of total power), representing static power dissipation when the gates are idle.
- **Internal power:** 5.47318 µW (82.52% of total power), representing active power consumed during internal node transitions within the standard-cell boundaries.
- **Switching power:** 950.632 nW (14.33% of total power), representing the power consumed when charging and discharging net capacitances.

Block/Category	Leakage (W)	Internal (W)	Switching (W)	Power %
Register (storage)	1.65826×10^{-7}	5.14986×10^{-6}	1.56738×10^{-7}	82.51%
Logic (combinational)	4.28900×10^{-8}	3.23326×10^{-7}	1.73490×10^{-7}	8.14%
Clock (buffers)	0.00000	0.00000	6.20405×10^{-7}	9.35%
Total	2.08716×10^{-7}	5.47318×10^{-6}	9.50632×10^{-7}	**100.00%**

Table 6: Post-synthesis power breakdown by block category.

Power dominance analysis: at the operating frequency of 131.072 kHz, register switching accounts for 82.51% of the power budget. Because the clock frequency is low, gate and interconnect switching activity is negligible, and the dominant power cost comes from toggling and holding the 476 output and state flip-flops. Both the area and power breakdowns show that this design is **register-dominated**, which matches the typical profile of a configuration and trim storage block rather than a compute-heavy datapath.

Task Division:

- **Erik Hartouni (Role C):** Responsible for executing the report-generation scripts in Design Compiler, extracting the timing paths, calculating the setup and hold slack margins, and compiling the timing analysis report.

- **Sina Hashemi (Role A):** Responsible for analyzing cell area utilization, checking the gate-count distribution between combinational and sequential logic, and optimizing the RTL files to minimize cell area.
- **Mohammadhossein Sabzalian (Role B) & Ali Behrad (Role D):** Supported the timing verification by reviewing the report outputs to ensure there were no setup or hold violations before finalizing the netlist.

Part 8 – Physical Design Hand-off

Scope of This Phase

Full physical implementation (floorplanning, placement, clock-tree synthesis, routing, and DRC/LVS sign-off) falls outside the 4-week RTL-to-gate-level scope defined by the project guideline: Week 4 calls only for the design database and setup scripts required for a downstream place-and-route hand-off, not for a completed layout. This section documents what was exported and prepared for that hand-off.

Design Database Export

The final stage of `syn_scr.tcl` (listing 3) exports, alongside the structural netlist and SDF, a complete Innovus-compatible design database via `write_design ... -innovus`. This database packages the mapped gate-level netlist together with its physical library views (LEF/technology data) so that a place-and-route tool can load the design without re-running synthesis.

> **Innovus Placement-and-Routing Setup:** A dedicated setup script, `otp_controller.invs_setup.tcl`, was prepared and included in the hand-off package. It configures the Innovus environment (technology and timing library references, the exported synthesis design database, and the same 131.072 kHz clock definition used during synthesis) so that floorplanning and placement can begin directly from the synthesized netlist without re-deriving constraints by hand.

Physical Design Hand-off Package

Hand-off Package Contents

- **Structural netlist:** `otp_controller_postsyn.v` (1,836 standard-cell instances, TSMC 180 nm).
- **Back-annotation file:** `otp_controller_postsyn.sdf` (MAXIMUM-corner delays).
- **Constraints snapshot:** `constraints.g`, a re-runnable record of the applied timing constraints.
- **Innovus design database:** exported via `write_design ... -innovus` for direct place-and-route load-in.
- **Innovus setup script:** `otp_controller.invs_setup.tcl`, configuring libraries, the design database, and clock definitions for placement and routing.

Why the OTP array is excluded: consistent with table 3 and DRS §2.1, the 256-bit OTP memory (`otp_sim_rom.v`) is a behavioral, simulation-only stand-in for a foundry-specific non-volatile memory macro. It was deliberately excluded from synthesis and from the physical hand-off package, since a real OTP macro is a licensed analog/mixed-signal

IP block dropped in directly at the physical-design stage rather than a synthesizable standard-cell netlist.

Task Division:

- **Erik Hartouni (Role C):** Finalized the synthesis export stage of `syn_scr.tcl`, verified the clean generation of the Innovus design database, and organized the hand-off directory structure.
- **Ali Behrad (Role D):** Assembled and verified the complete hand-off package (RTL, testbench, synthesis, and physical-design deliverables) for system integration submission.
- **Sina Hashemi (Role A) & Mohammadhossein Sabzalian (Role B):** Prepared the `otp_controller.invs_setup.tcl` placement-and-routing setup script, carrying over the library references and the 131.072 kHz clock definition from the synthesis constraints.

Part 9 – Comparative Functional vs. Post-Synthesis Simulation

Verification Baseline Strategy

To establish a reliable basis for comparison, the functional regression suite (`func_sim.tcl`) was re-run unmodified against the RTL design before attempting GLS. This verified that the simulation environment – including compile options, parameter directories, and verification scripts – remained stable. Establishing this baseline ensures that any subsequent discrepancies observed during post-synthesis simulations are caused by synthesis mismatches or timing violations, rather than testbench or environment issues.

Standard Delay Format (SDF) Back-Annotation Analysis

During gate-level simulation, worst-case timing delays were back-annotated from the Standard Delay Format (`otp_controller_postsyn.sdf`) file. The simulator's elaboration tool successfully annotated the timing paths:

SDF Annotation Coverage

- **Path delays:** 4,979 of 4,979 (100.00%) gate-to-gate path delays were successfully annotated.
- **Width checks:** 1,428 of 1,428 (100.00%) pulse-width timing checks were annotated.
- **Setup/hold (`$setuphold`) checks:** 1,080 of 1,556 (69.41%) checks were annotated.

The remaining 30.59% of unannotated setup/hold timing checks correspond to the RECREM (Recovery/Removal) timing checks defined on the asynchronous reset pins (`RN`) of standard sequential cells (such as `DFFRHQX1` and `SDFFRHQX1`). Because the asynchronous reset net `rst_n` was not constrained as an ideal clock net during synthesis, Genus did not generate recovery/removal timing values for these pins. **Because these timing checks belong to library primitives that are not active during the main execution phase, and because zero SDF annotation errors were reported, these warnings did not affect the timing or functional correctness of the design.**

How to Perform the Comparative Analysis

The pre-synthesis (RTL) and post-synthesis (gate-level) runs were compared directly against one another using their respective `xrun.log` execution logs and SimVision waveform captures. Both simulations were driven with the identical stimulus sequence and golden reference checks, so that any divergence in the scoreboard results or in the final simulation timestamp would immediately flag a synthesis-induced mismatch. The two runs were confirmed to reach an identical simulation-completion time of 17,238,606,705 ps, and the register-level waveforms of both runs were overlaid in SimVision to visually confirm bit-exact agreement at every sampled configuration output.

Task Division:

- **Ali Behrad (Role D):** Set up the gate-level simulation flow, compiled the TSMC cell primitives with the structural netlist, ran the back-annotated timing simulation, and exported the `postsyn_sim_xrun.log` files.
- **Mohammadhossein Sabzalian (Role B):** Analyzed the SDF back-annotation warnings, compared the pre-synthesis and post-synthesis waveforms, and verified that both runs completed at exactly 17,238,606,705 ps.
- **Sina Hashemi (Role A) & Erik Hartouni (Role C):** Supported this phase by delivering a clean, timing-closed netlist (`otp_controller_postsyn.v`) and an error-free SDF file (`otp_controller_postsyn.sdf`).

Part 10 – Circuit Optimization

Logical and Protocol Optimizations

FSM Next-State Look-Ahead Restructuring

During cross-group verification with Group 7, a timing discrepancy was discovered in the FSM control path when evaluated under strict address-assertion checkers.

FSM Timing Discrepancy and Fix

- **The issue:** the initial FSM design updated the internal address counter (`addr`) and transitioned states "one cycle slower" (with a one-clock-cycle delay). Because the register updates for the address counter and state transitions were sequentially offset, the controller spent an extra, redundant clock cycle in the active read states, executing 257 read cycles instead of the specified 256.
- **The correction:** the FSM next-state transition logic and the sequential address-counter increment path were redesigned. By restructuring the combinational look-ahead paths, state transitions and address changes were synchronized to occur concurrently on the exact same clock edge. This successfully restricted the read phase to exactly 256 cycles, ensuring protocol compliance.

Physical and Compiler-Driven Optimizations

Sequential Register Merging and Pruning

During the incremental optimization stage (`syn_opt`) in Cadence Genus, high-effort register merging was performed:

- The tool analyzed the registers and executed equivalent sequential merging (GLO-42).
- This pruned three redundant sequential instances, reducing the active register count from 479 down to **476 active physical registers** without altering design functionality.
- This optimization reduced the physical sequential area, lowering static leakage power.

Parallel Galois-Style CRC-16 Engine

Instead of utilizing a multi-stage serial feedback shift register, the CRC-16 CCITT module is implemented using a Galois parallel feedback XOR tree:

Parallel CRC-16 Engine Results

- This architecture processes the incoming bitstream and computes the checksum on-the-fly in a single clock cycle.
- The entire parallel engine was synthesized into just 6 cells, consuming a physical footprint of only 103.118 μm^2 (less than 0.2% of the total design area).
- This design keeps the critical path delay low (2.727 ns), maintaining a setup timing slack of $+7.626\,\mu$s.

Task Division:

- **Sina Hashemi (Role A):** Redesigned the FSM combinational look-ahead paths to resolve the 257-cycle timing discrepancy, synchronized the address-counter increments, and designed the parallel Galois CRC engine.
- **Erik Hartouni (Role C):** Set up the incremental synthesis run (`syn_opt`) in Genus, analyzed the area and register-merging logs, and verified the reduction of redundant sequential registers.
- **Mohammadhossein Sabzalian (Role B) & Ali Behrad (Role D):** Conducted cross-group verification with Group 7, identified the FSM timing discrepancy, and verified the optimized RTL design using both the home and exchanged testbenches.